

THREADS

Messaging



Messaging Project Definition

(Spring 2026)

Version 1.2

1. Introduction

The second stage of our OS development project adds to the kernel's core functionality through the implementation of mailboxes, a core IPC mechanism, and through the initial handling of device I/O and system calls. This is the Messaging project.

1.1. Objectives and Scope

The objective of this document is to specify requirements for the Messaging development project. Included is a specification for the required functions (e.g. the Messaging Application Programming Interface (API)) that must be implemented for this part of the kernel's development.

This document combined with the course's lecture and presentation materials define the totality of the Messaging project.

1.2. Target Audience

The document is intended for students taking advanced operating systems courses at the graduate or undergraduate level.

2. Messaging Layer

The Messaging project incrementally adds functionality to our kernel through the implementation of a mailbox system and through initial handling of system calls and I/O interrupts. The Messaging system builds on what was done in the Scheduler project and uses the Scheduler interface for core kernel operations. See Figure 1.

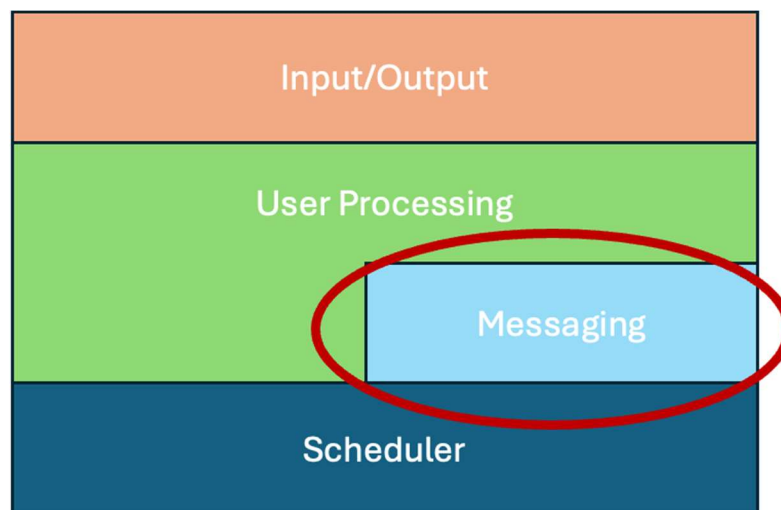


Figure 1 The Messaging feature relies on the Scheduler interface.

Like the Scheduler project, we define an interface for the Messaging system that, combined with the Scheduler, provide a more full-featured kernel that includes process management and messaging. Future layers of the architecture rely on the Messaging system for various IPC functions.



Important: All processing done in the Messaging is performed in **kernel** mode. This mode must be enforced to prevent the invocation of a Messaging function in **user** mode.

2.1. Messaging Startup: SchedulerEntryPoint()

During the Scheduler project, the entry point for all our test cases was a process that begins with a call to the **SchedulerEntryPoint** function. This process is started at the end of bootstrap processing and represents the starting point of the whole kernel. The **SchedulerEntryPoint** is where the Messaging project begins and is the place where we 1) initialize the parts of the kernel that manage messaging and system calls and 2) start the **MessagingEntryPoint** process. As with the Scheduler phase, the **MessagingEntryPoint** is used to launch each of the testcases that exercise the Messaging project functionality.

To begin, students are provided with a THREADS-Messaging Starter Project that begins with a partially implemented **SchedulerEntryPoint()** function. The Starter Project lets students immediately begin working through the initialization needed for the messaging system.

Before interrupts are enabled, **SchedulerEntryPoint** must ensure the following conditions are met:

- The processor is in kernel mode.
- The mailbox table and the global mail slot pool are initialized. A device mailbox has been allocated for each of the **THREADS_MAX_DEVICES** device slots. The clock device (**THREADS_CLOCK_DEVICE_ID**) uses a zero-slot mailbox. All other device slots use slotted mailboxes.
- Each physical device (disk0, disk1, term0, term1, term2, term3) has been registered with THREADS via **device_initialize**, and its handle has been associated with the corresponding device mailbox.
- The three interrupt handlers described in Section 2.2 have been registered in the interrupt handler table.
- Every entry in the **systemCallVector** array points to the **nullsys** error handler.

After enabling interrupts, **SchedulerEntryPoint** spawns **MessagingEntryPoint** at **HIGHEST_PRIORITY** with a stack size of $4 * \text{THREADS_MIN_STACK_SIZE}$, then blocks on **k_wait** until the test process exits. When **k_wait** returns, **SchedulerEntryPoint** calls **k_exit(0)**.

2.2. Interrupt Handlers

THREADS delivers interrupts through a table of handler function pointers obtained via `get_interrupt_handlers`. All handlers must conform to the `interrupt_handler_t` signature defined in `THREADSLib.h`. Three handlers are required. The index constants used to register them are defined in `THREADSLib.h`.

2.2.1. Clock Handler — `THREADS_TIMER_INTERRUPT`

THREADS fires a timer interrupt approximately every 20 milliseconds. The clock handler must:

- Maintain a tick count. On every 5th tick, send a non-blocking message to the clock device mailbox and reset the count. If `mailbox_send` returns `-1` or `-5`, log the error and call `stop(1)`.
- Call `time_slice()` on every tick.

2.2.2. I/O Handler — `THREADS_IO_INTERRUPT`

THREADS fires an I/O interrupt when a disk or terminal device completes an operation. The device parameter identifies which device generated the interrupt; status carries the device's status register value.

The I/O handler must send the status value to the identified device's mailbox using a non-blocking `mailbox_send`. If `mailbox_send` returns `-1` or `-5`, log the error and call `stop(1)`.

2.2.3. System Call Handler `THREADS_SYS_CALL_INTERRUPT`

THREADS fires this interrupt when user-mode code invokes `system_call()`. The `pArgs` parameter points to a `system_call_arguments_t` structure containing the `call_id` and arguments.

The system call handler must validate the device identifier (`THREADS_SYSTEM_CALL_ID`), verify `pArgs` is not `NULL`, verify `call_id` is in range, and dispatch to the corresponding entry in `systemCallVector`. System call functions are registered in later project phases; the handler provides the dispatch infrastructure.

3. Messaging Overview

Inter-Process Communication (IPC) is a cornerstone of a kernel's operation. In a modern operating system, there are many mechanisms available for processes to communicate with one another and synchronize activities. These mechanisms include semaphores, mutexes, pipes, sockets, and more.

In our THREADS kernel, we begin support for IPC with a mail-based system for communication. This mail-based system enables processes to both send and receive messages and is modelled after a physical mail delivery system where mail (a message)

is placed in a mailbox and the mail is eventually received (“picked up”) by the recipient when it is removed from the mailbox.

The Messaging mailbox system has the following important requirements:

- Mailboxes have a set number of mail slots where each slot can hold one message.
- Mail must be received in the order it is sent.
- Mailboxes may be either blocking or non-blocking.
- In blocking mailboxes, the following rules apply:
 - A sending process must block when sending a message until a mail slot is available to hold the message in the mailbox
 - A receiving process must block if there is no mail in the mailbox
- In non-blocking mailboxes, both the sender and the receiver return immediately regardless of whether mail was delivered or received. (An error code is used to indicate successful delivery or reception.)
- Mailboxes can be created with zero slots (Zero-slot mailboxes).

These rules may appear relatively straightforward, but the implementation of such a mailbox system becomes complex in an environment where many processes of different priorities are running at the same time.



Important: *Maintaining order of mail must be maintained regardless of the priority of the sending and receiving processes.*

Mailboxes are identified by a unique id. Processes can create and free mailboxes. The mailbox is not “owned” by a particular process; that is, a process can create a mailbox, and a different process can release it, possibly after the creating process has exited.

Zero-slot mailboxes are a unique type of mailbox and are intended for synchronization between a sender and a receiver. These mailboxes have no slots and are used to pass messages directly between processes; they act much like a signal. The sender is blocked until a receiver collects the message, or the receiver will be blocked until the sender sends the message. The non-blocking operations will not block on zero-slot mailboxes but may succeed if a process is currently blocked as a sender or receiver.

You must ensure that messages are delivered in the order sent and are received in the order the receive function is called. In general, the slot mechanism helps to ensure that this order is maintained. However, process priority must be considered here. For example, consider a low priority process that blocks on a receive, followed by a high priority process that subsequently runs and calls receive on the same mailbox. When a message is sent to this mailbox, the message must be received by the low priority process since it is at the head of the queue. The higher priority process must wait for another message. When two messages are sent to this same mailbox, the first message sent must be received by the low priority process and the second message is received by the high priority process.

There are three constants defined in the provided Messaging.h header file:

1. MAXMBOX specifies the maximum number of mailboxes.

2. MAXSLOTS specifies the maximum number of mailbox slots that can be in use at any one time. Note that this is the number of active messages being held in mailboxes. Thus, you can create mailboxes whose total slots are more than MAXSLOTS so long as there are not more than MAXSLOTS actual messages in the system at one time.
3. MAX_MESSAGE specifies the largest possible size of one message in bytes.

4. Messaging Interface

The Messaging interface defines a set of API functions that future iterations of the OS, with more functionality, can use and leverage the mail system. These functions must be implemented in your Messaging project. The Starter Project contains prototypes for all these functions and a partial implementation for some of them.

4.1.1. mailbox_create

int mailbox_create (int slot_count, int slot_size);			
Description	The mailbox_create function creates a mailbox with a specified number of slots, <i>slot_count</i> , and a maximum message size, <i>slot_size</i> .		
Arguments	<i>slot_count</i>	The number of slots available for the mailbox to hold message. Value may be zero to 2500.	
	<i>slot_size</i>	The maximum size, in bytes, of messages in the mailbox. The value must be between 0 and 256.	
Return Value	int	Upon success, the function returns the mailbox id of the newly created mailbox. The function should return an error code in an error occurs:	
		Return Value	Result
		>= 0	The identifier of the created mailbox.
		-1	A parameter is missing or invalid.

4.1.2. mailbox_free

int mailbox_free(int mailbox_id);		
Description	The mailbox_free closes a previously created mailbox. Any process waiting (blocked processes) on the mailbox should be signaled with k_kill and unblocked. A waiting process that has been signaled must detect the closed mailbox when it is unblocked.	
Arguments	<i>mailbox_id</i>	The id of the mailbox to close.

Return Value	int	The return value must be one of the following error codes:	
		Return Value	Result
		0	The message was successfully closed.
		-1	An invalid parameter was passed to the function.
		-5	The process was signaled while closing the mailbox.

NOTES:

It is possible to free a mailbox where there are multiple processes either waiting to send or waiting to receive messages. The `mailbox_free` function is responsible for unblocking these processes and signaling them. This must be done in a way that a process does not access a fully-freed mailbox in the mailbox table after it is unblocked.

4.1.3. mailbox_send

int mailbox_send (int mailbox_id, void *message, int message_size, bool block);		
Description	The <code>mailbox_send</code> function's purpose is to deliver mail to the mailbox specified by <code>mailbox_id</code>. The <code>message</code> parameter points to the message to deliver, and the <code>message_size</code> parameter specifies the size of the message. The <code>block</code> parameter specifies that the function must block the calling processes until there is an available slot in the mailbox to deliver the message.	
Arguments	<i>mailbox_id</i>	The identifier of the mailbox to deliver the message to.
	<i>message</i>	A pointer to the message.
	<i>message_size</i>	The size, in bytes, of the message pointed to by <i>message</i> .
	<i>block</i>	If false, the function returns immediately regardless of whether there is an available slot in the mailbox. If true, the function must block until a slot in the mailbox becomes available.

Return Value	<i>int</i>	The return value must be one of the following error codes:	
		Return Value	Result
		0	The message was successfully delivered to the mailbox.
		-1	An invalid parameter was passed to the function.
		-2	The operation could not complete without waiting, and non-blocking mode was specified.
		-5	The process was signaled while waiting.

NOTES:

If there is an empty slot in the mailbox, the message can be delivered to the mailbox immediately and the send function returns without blocking. If the **block** flag is false, the function never blocks waiting to send a message.

Make sure to treat the message as binary data. Use the **memcpy** function to copy the data into the mailbox slot. *Do not use **strcpy** to copy messages.*

If the system is out of mailbox slots, print an appropriate error message and call **stop(1)**.

Messages can be of zero length.

4.1.4. mailbox_receive

int mailbox_receive(int mailbox_id, void *message, int max_message_size, bool block);		
Description	The mailbox_receive function's purpose is to receive mail from the mailbox specified by <i>mailbox_id</i>. The <i>message</i> parameter points to the message to buffer where that the message is copied into, and the <i>max_message_size</i> parameter specifies the size of the buffer, which is the maximum size of the received message. The <i>block</i> parameter specifies that the function must block the calling processes until there is a message has been delivered to the mailbox.	
Arguments	<i>mailbox_id</i>	The identifier of the mailbox to deliver the message to.
	<i>message</i>	A pointer to the message buffer.
	<i>max_message_size</i>	The size, in bytes, of the buffer pointed to by <i>message</i> .

	<i>block</i>	If false, the function returns immediately regardless of whether there is mail in the mailbox. If true, the function must block until a message has been delivered to the mailbox.										
Return Value	<i>int</i>	The return value must be one of the following error codes: <table><tr><th>Return Value</th><th>Result</th></tr><tr><td><code>>=0</code></td><td>The size of the received message.</td></tr><tr><td><code>-1</code></td><td>An invalid parameter was passed to the function.</td></tr><tr><td><code>-2</code></td><td>The operation could not complete without waiting, and non-blocking mode was specified.</td></tr><tr><td><code>-5</code></td><td>The process was signaled while waiting.</td></tr></table>	Return Value	Result	<code>>=0</code>	The size of the received message.	<code>-1</code>	An invalid parameter was passed to the function.	<code>-2</code>	The operation could not complete without waiting, and non-blocking mode was specified.	<code>-5</code>	The process was signaled while waiting.
Return Value	Result											
<code>>=0</code>	The size of the received message.											
<code>-1</code>	An invalid parameter was passed to the function.											
<code>-2</code>	The operation could not complete without waiting, and non-blocking mode was specified.											
<code>-5</code>	The process was signaled while waiting.											

NOTES:

If the **block** flag is false, the function never blocks waiting to receive a message.

The caller of this function is responsible for providing a buffer where the message is placed when it's received. *Messages can be of zero length!*

4.1.5. wait_device

int wait_device (char *device_name, int *status);		
Description	The <code>wait_device</code> function is used in handling device I/O requests by waiting for a signal that is generated in by an interrupt. The kernel's interrupt system is invoked when • a device finishes an I/O request, • an error occurs in a device, or • the timer interrupt is fired. The function returns the devices status, which is a device-specific value that indicates the completion status of an I/O operation. The timer event is handled differently from a device operation in that the clock device should be signaled every 100 milliseconds.	
Arguments	<i>device_name</i>	The name of the device to wait for.
	<i>status</i>	A device-specific status value, see the THREADS documentation for details on the status value.

Return Value	<i>int</i>	The return value must be one of the following error codes: <table><tr><th>Return Value</th><th>Result</th></tr><tr><td>0</td><td>The device was signaled.</td></tr><tr><td>-1</td><td>An invalid parameter was passed to the function.</td></tr><tr><td>-5</td><td>The process was signaled while waiting.</td></tr></table>	Return Value	Result	0	The device was signaled.	-1	An invalid parameter was passed to the function.	-5	The process was signaled while waiting.
Return Value	Result									
0	The device was signaled.									
-1	An invalid parameter was passed to the function.									
-5	The process was signaled while waiting.									

NOTES:

The device's identifier is obtained using the THREADS function **device_handle**. Check the return value from this function and call **stop(1)** if the value is out of range.

5. System Calls

The provided Messaging Starter project contains initial handling capabilities for system call management. The system call vector is defined at the top of Messaging.c and each entry in the vector is set to a function titled nullsys.

6. The Scheduler Functions

In this layered design, access to global values in the lower layers is generally prohibited. Therefore, the Scheduler library only exposes the following functions for your Messaging project: **k_spawn**, **k_join**, **k_wait**, **k_exit**, **k_kill**, **signaled**, **k_getpid**, **display_process_table**, **block**, **unblock**, **get_start_time**, and **time_slice** are available for you to use. No other Scheduler functions or variables are accessible. For example, you cannot call the dispatcher directly, reference **runningProcess** or any other global value in the Scheduler.